MySQL - HackerRank Notebook

Pham Dang Hoang Thien

February 7, 2026

Student, Ho Chi Minh City, Vietnam
e-mail: hoangthien312006@gmail.com

Contents

1 Introduction to MySQL

2 Basic SQL Queries

2.1 Triangle Classification Query

Write a query identifying the type of each record in the `TRIANGLES` table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with 3 sides of equal length.
- **Isosceles**: It's a triangle with 2 sides of equal length.
- **Scalene**: It's a triangle with 3 sides of differing lengths.
- **Not A Triangle**: The given values of A, B, and C don't form a triangle.

Input Format:

Column	*Type*
A	*Integer*
B	*Integer*
C	*Integer*

The `TRIANGLES` table contains three columns: A, B, and C, representing the lengths of the sides of a triangle.

Sample Input:

A	B	C
20	20	23
20	20	20
20	21	22
13	14	30

Sample Output:

```
Isosceles
Equilateral
Scalene
Not A Triangle
```

Solution Hint

Main idea: Use a `CASE` expression to classify each triangle in this order:

1. Check **Not A Triangle** first using the triangle inequality:

$$A + B \leq C \text{ or } A + C \leq B \text{ or } B + C \leq A$$

2. If valid, check **Equilateral**: `A = B AND B = C`
3. Check **Isosceles**: exactly two sides are equal
4. Otherwise, it is **Scalene**

Required syntax / clauses:

- `CASE WHEN ... THEN ... ELSE ... END`
- Logical operators: `AND`, `OR`
- Comparison operators: `=`, `<=`

SQL Query

```
SELECT
  CASE
    WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A
        Triangle'
    WHEN A = B AND B = C THEN 'Equilateral'
    WHEN A = B OR B = C OR A = C THEN 'Isosceles'
    ELSE 'Scalene'
  END AS triangle_type
FROM TRIANGLES;
```

Why this condition order?

- Put `Not A Triangle` first to filter out invalid side combinations.
- Then classify only valid triangles as equilateral, isosceles, or scalene.

2.2 Revising Aggregations

Input Format: The CITY table is described as follows:

Fields	*Type*
ID	*Number*
NAME	*VARCHAR2(17)*
COUNTRYCODE	*VARCHAR2(3)*
DISTRICT	*VARCHAR2(20)*
POPULATION	*Number*

2.2.1 The Count Function

Query a count of the number of cities in CITY having a Population larger than 100,000.

Solution Hint

The query should count the number of rows in the CITY table where the POPULATION is greater than 100,000.
COUNT(*) is used to count all rows that meet the specified condition in the WHERE clause. AS ***city_count*** renames the output column for clarity.

SQL Query

```
1 SELECT COUNT(*) AS city_count
2 FROM CITY
3 WHERE POPULATION > 100000;
```

2.2.2 The Sum Function

Query the total population of all cities in CITY where District is California.

Solution Hint

The query should sum the POPULATION column in the CITY table where the DISTRICT is 'California'.
SUM(POPULATION) is used to calculate the total population of rows that meet the specified condition in the WHERE clause. AS ***total_population*** renames the output column for clarity.

SQL Query

```
SELECT SUM(POPULATION) AS total_population
FROM CITY
WHERE DISTRICT = 'California';
```

2.2.3 Averages

Query the average population of all cities in CITY where District is California.

Solution Hint

The query should calculate the average of the POPULATION column in the CITY table where the DISTRICT is 'California'.
AVG(POPULATION) is used to calculate the average population of rows that meet the specified condition in the WHERE clause. AS ***average_population*** renames the output column for clarity.

SQL Query

```
SELECT AVG(POPULATION) AS average_population
FROM CITY
WHERE DISTRICT = 'California';
```

2.2.4 Average Population

Query the average population for all cities in CITY, rounded down to the nearest integer.

Solution Hint

The query should calculate the average of the POPULATION column in the CITY table and round it down to the nearest integer.

- Use ***AVG(POPULATION)*** to get the average population.
- Use ***ROUND(...)*** to round the average population to the nearest integer.
- AS ***average_population*** renames the output column for clarity.

SQL Query

```
SELECT ROUND(AVG(POPULATION)) AS average_population
FROM CITY;
```

2.2.5 Japan Population

Query the total population of cities in CITY where CountryCode is JPN.

Solution Hint

The query should sum the POPULATION column in the CITY table where the COUNTRYCODE is 'JPN'.
SUM(POPULATION) is used to calculate the total population of rows that meet the specified condition in the WHERE clause. AS ***total_population*** renames the output column for clarity.

SQL Query

```
SELECT SUM(POPULATION) AS total_population
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

2.2.6 Population Density Difference

Query the difference between the maximum and minimum populations in CITY.

Solution Hint

The query should calculate the difference between the maximum and minimum values of the POPULATION column in the CITY table.

- Use ***MAX(POPULATION)*** to get the maximum population.
- Use ***MIN(POPULATION)*** to get the minimum population.
- AS ***pop_diff*** renames the output column for clarity.

SQL Query

```
SELECT MAX(POPULATION) − MIN(POPULATION) AS pop_diff
FROM CITY;
```

2.3 The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's 0 key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary. Write a query calculating the amount of error (i.e.: *actual-miscalculated* average monthly salaries), and round it up to the next integer.

Input Format

The `EMPLOYEES` table is described as follows:

Column	*Type*
ID	*Integer*
Name	*String*
Salary	*Integer*

Note: Salary is per month.

Constraints

$$1000 < \text{Salary} < 10^5$$

Sample Input

ID	*Name*	*Salary*
1	Kristeen	1420
2	Ashley	2006
3	Julia	2210
4	Maria	3000

Sample Output

```
2061
```

Explanation

The table below shows the salaries without zeros as they were entered by Samantha:

ID	*Name*	*Salary*
1	Kristeen	142
2	Ashley	26
3	Julia	221
4	Maria	3

Samantha computes an average salary of 98.00. The actual average salary is 2159.00.

The resulting error between the two calculations is

$$2159.00 - 98.00 = 2061.00$$

Since it is equal to the integer 2061, it does not get rounded up.

Solution Hint

The query should calculate the difference between the actual average salary and the miscalculated average salary (with zeros removed). The difference should be rounded up to the next integer.

- ***AVG(Salary)*** is used to calculate the actual average salary.
- ***REPLACE(CAST(Salary AS CHAR), '0', '')*** is used to remove all zeros from the salary by converting it to a string, replacing '0' with an empty string, and then casting it back to an unsigned integer. /dễ hiểu là bỏ hết số 0 trong salary (mô phỏng bàn phím hỏng) đi rồi tính trung bình.
- ***AS UNSIGNED*** ensures that the modified salary is a positive integer for the average calculation. /nhận giá trị từ 0 trở lên.

```
SELECT CEIL(
        AVG(Salary) − AVG(CAST(REPLACE(CAST(Salary AS CHAR),
            '0', '') AS UNSIGNED))
    ) AS error
FROM EMPLOYEES;
```